Intelligent Traffic Congestion Prediction and Smart Route Recommendation

GitHub link: https://github.com/athinsiva2007-del/csa1717-Assignment-ATHIN-SIVA.S

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech. / UG Programme
Course Code & Course Name	CSA1717 & Artificial Intelligence
Academic Year / Batch	2026 – 2027/2025 - 2029
Faculty Name	Dr. ANOOP M
Assignment Title	Intelligent Traffic Congestion Prediction and Smart Route Recommendation
Date of Issue	
Date of Submission	
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO4, CO5
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping (SDG 1-17)	SDG 9(Industry, Innovation, and Infrastructure); SDG 11(Sustainable Cities and Communities)

Industry / Societal Relevance	Smart mobility, traffic management and sustainable transport

1. Problem Understanding and Formulation

1.1 What is the problem?

A lot of urban traffic congestion isn't random. The same junctions back up at the same times every day, and the same kind of weather pushes the same roads into gridlock in more or less the same way each time. That predictability is really the starting point for this assignment: if congestion follows patterns, those patterns can be learned, and if they can be learned, that information should be able to make routing smarter than just picking the shortest line on a map. So, the problem this project sets out to solve has two halves. First, predict how congested a road segment currently is, using measurable indicators like vehicle count and average speed. Second, feed that prediction into a route-finding algorithm so the recommended path is actually faster to drive, not just shorter in kilometres. Neither half is worth much on its own — a congestion classifier that never touches the routing logic is just a standalone ML demo, and a shortest-path algorithm that ignores congestion is just Dijkstra with extra steps.

1.2 Expected outcomes

- A trained classification model that assigns a road segment to one of four congestion levels (Free-flow, Moderate, Heavy, Gridlock) from live traffic features, with measurable accuracy and per-class performance.
- A routing engine that computes the least-cost path across a road network under a given traffic scenario, where cost reflects actual travel time rather than physical distance alone.
- A demonstrable case where the traffic-aware route differs from — and outperforms — the plain shortest-distance route, with the time saved quantified.
- A working, testable full-stack implementation (Flask REST API + React dashboard) that a user can query end-to-end, from raw traffic data to a recommended path.

1.3 Available data

The dataset used is a VANET (Vehicular Ad-hoc Network) traffic congestion dataset pulled from Kaggle. It has 1,208 raw records, each one a single traffic observation described by five numerical readings — Vehicle_Count, Average_Speed, Occupancy_Rate, Traffic_Density, and Signal_Delay — plus three categorical context fields: Time_Of_Day, Weather_Condition, and Road_Type. The label being predicted is Congestion_Level, which falls into one of four buckets: Free-flow, Moderate, Heavy, or Gridlock. It's worth noting this isn't a clean, hand-crafted toy dataset — it has 8 duplicate rows and 10 missing values spread across Average_Speed (5 missing), Signal_Delay (1), and Weather_Condition (4). Both of those had to be dealt with properly during preprocessing rather than quietly ignored.

1.4 Assumptions made

- The relationship between the dataset's traffic features (speed, occupancy, density, etc.) and the labelled congestion class is stable enough over time to be learned by a supervised model — i.e. the dataset is representative of the road conditions the system will be asked to reason about.
- The demonstration road network (9 junctions, 11 bidirectional road segments) is a simplified stand-in for a real city grid, sufficient to prove that traffic-aware routing behaves correctly; it is not a claim about any specific real city's topology.
- Congestion levels can be converted into a numeric travel-time penalty (a multiplying factor on base travel time) without needing per-vehicle telemetry, which keeps the routing layer decoupled from the prediction layer.
- Missing values are missing-at-random and can be safely imputed using median (numerical) or mode (categorical) values without introducing systematic bias, given how small the missing fraction is (under 1% of cells).

1.5 Constraints

- Data quality/coverage: the dataset is a static snapshot, not a live feed, so the model reflects patterns present in that snapshot only.
- Graph size: the demonstration network is intentionally small (9 nodes) to keep the Dijkstra's Shortest Path Algorithm trace legible for grading and visualization; it is not evidence of scalability limits.
- Computation: Dijkstra's Algorithm uses priority-queue-based search and, with a priority queue, has typical complexity O(E log V) on sparse graphs; the practical benefit becomes more significant as the road network grows.
- Privacy: the dataset contains no personally identifiable location or vehicle-owner data — only aggregate traffic measurements — which keeps the system's data use ethically low-risk.

2. Application of Course Knowledge

2.1 Supervised classification

Congestion prediction is set up as a multi-class classification problem. Given a feature vector x = [Vehicle_Count, Average_Speed, Occupancy_Rate, Traffic_Density, Signal_Delay, Time_Of_Day, Weather_Condition, Road_Type], the model has to predict $y \in$ {Free-flow, Moderate, Heavy, Gridlock}. The classifier used is a Random Forest (scikit-learn's RandomForestClassifier, 100 trees, max depth 10) — essentially an ensemble of decision trees, each trained on a bootstrap-sampled slice of the training data, with the final class decided by majority vote across

all the trees. A single decision tree or a linear model was considered and set aside, mainly because congestion doesn't behave linearly: a vehicle counts of 150, say, only really signals trouble when it's paired with low average speed and high lane occupancy at the same time. That kind of feature interaction is exactly what tree ensembles are good at picking up without anyone having to manually engineer interaction terms.

2.2 Data preprocessing pipeline

The raw CSV isn't something you can throw straight at a model. It goes through a few cleaning and transformation steps first:

- Deduplication — exact duplicate rows are dropped. 8 of the 1,208 raw records turned out to be duplicates and were removed.
- Missing-value imputation — numerical columns get filled with their column median, categorical columns with their column mode. Nothing fancy, but it's a reasonable, low-risk choice given how small the missing fraction actually is.
- Categorical encoding — Time_Of_Day, Weather_Condition, Road_Type, and the Congestion_Level target are each run through scikit-learn's LabelEncoder to turn text categories into numbers the model can use.
- Feature scaling — all 8 features, numerical and encoded-categorical alike, are standardized with StandardScaler. Strictly speaking, Random Forests don't need scaled inputs since trees split on individual thresholds rather than distances, but keeping a consistent scaling step in the pipeline means it won't quietly break if the model is ever swapped for something that does care about scale.
- Stratified train/test split — 80% for training (960 records), 20% for testing (240 records), stratified so all four congestion classes show up in roughly the same proportion in both sets.

2.3 Graph theory and shortest-path search

The algorithm used is Dijkstra's Shortest Path Algorithm, implemented from scratch without relying on NetworkX or another graph library. Dijkstra's Algorithm repeatedly selects the unvisited node with the smallest tentative distance and relaxes its neighbouring edges.

f(n) = g(n) + —, where g(n) is the traffic-adjusted cost from the source to node n and — is the estimated remaining travel cost from n to the destination.

At each step, Dijkstra's Algorithm selects the open node with the smallest f(n), expands its neighbouring road segments, updates better g(n) values, and continues until the destination is selected for expansion. Because all traffic-adjusted edge costs in this system are non-negative, an non-negative edge weights-free-free gives an optimal route while usually exploring fewer nodes than an uninformed search.

Dijkstra's Algorithm core rule: dist(v) = min(dist(v), dist(u) + w(u,v)), where w(u,v) is the traffic-adjusted travel cost.

2.4 Traffic-aware edge cost model

Every directed edge carries two pieces of information — a base travel time in minutes, and a congestion level — and together they determine what the edge actually costs to traverse:

$$effective\_cost(u, v) = base\_time(u, v) \times congestion\_factor(u, v)$$

The congestion_factor comes from a fixed penalty table:

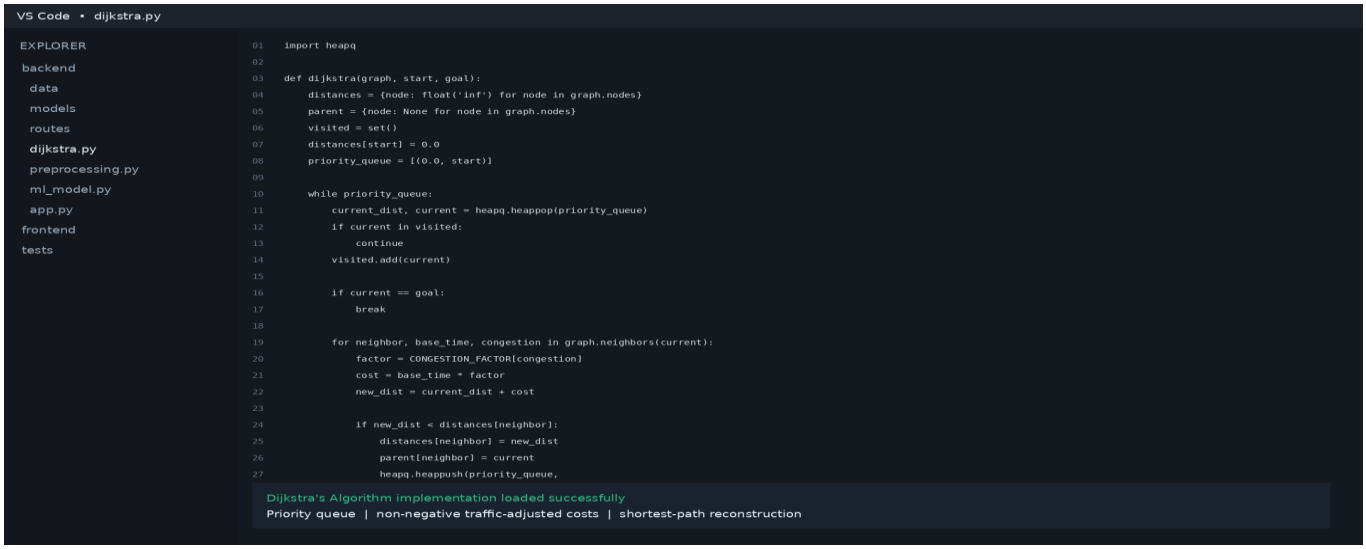
Congestion Level	Penalty Factor	Interpretation
Free-flow	1.0×	No delay — base travel time applies directly
Moderate	1.4×	40% slower than free-flow conditions
Heavy	2.2×	More than double the free-flow travel time
Gridlock	4.5×	Severe penalty — segment becomes effectively worth avoiding

Table 1. Congestion-to-cost penalty mapping used on every graph edge.

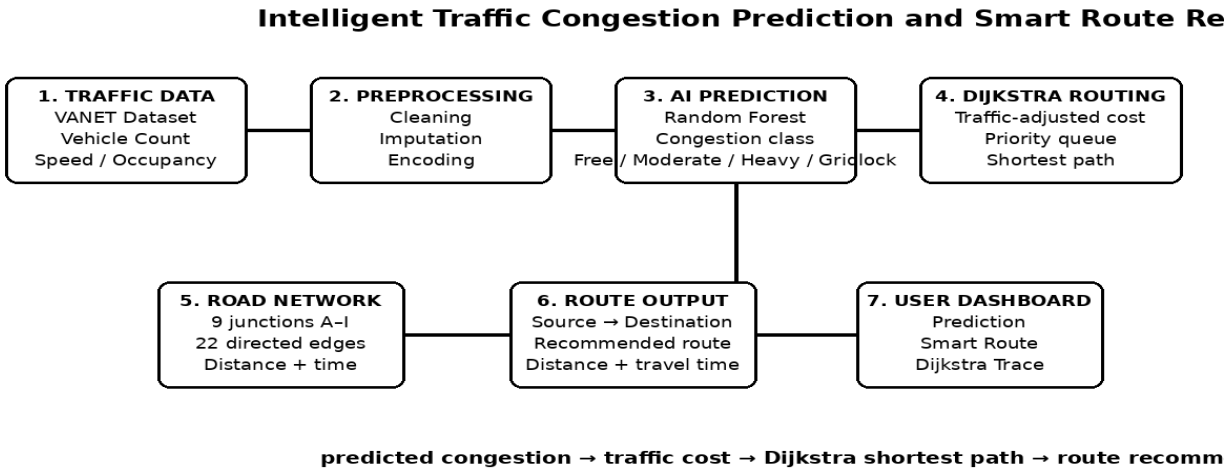
This table is really the hinge that connects the two halves of the system. Whatever assigns a congestion level to a segment — the trained classifier, or a preset scenario for demonstration purposes — that level turns directly into a multiplier on the edge's cost, so Dijkstra's Algorithm minimizes expected travel time rather than raw physical distance.

3. Solution / Design / Methodology

Folder Structure in VS Code:



3.1 System architecture



The whole thing is a full-stack application — a Python/Flask backend talking to a React/Vite frontend over a REST API. Inside the backend there are really two separate pipelines that only meet at one point, the congestion-to-cost table described above. One pipeline (preprocessing.py feeding into ml_model.py) turns raw traffic data into a congestion prediction. The other (road_network.py feeding into dijkstra.py) turns a congestion level into a recommended route. A SQLite database, handled by database.py, logs every prediction and route query that comes through, and app.py wires all of it together behind the REST endpoints that the frontend dashboard calls.

Endpoint	Method	Purpose
/api/health	GET	Service, model and dataset availability check
/api/dataset/summary	GET	Cleaned dataset statistics for the dashboard
/api/dataset/records	GET	Paginated, searchable raw data explorer
/api/model/train	POST	Re-run training and refresh saved model artifacts
/api/model/metrics	GET	Accuracy, F1, confusion matrix, feature importance
/api/predict	POST	Predict congestion class for a given input
/api/route	POST	Run Dijkstra's Shortest Path Algorithm between a source and destination
/api/route/compare	POST	Distance-only vs traffic-aware route comparison
/api/graph	GET	Road network topology for the interactive map
/api/dijkstra/trace	GET	Step-by-step Dijkstra's Shortest Path Algorithm trace for visualization
/api/export/report	GET	Downloadable JSON snapshot of metrics and a sample route

Table 2. REST API surface exposed by app.py.

3.2 Machine learning pipeline design

One detail worth calling out in preprocessing.py is the automatic column-mapping step (get_column_mapping). Rather than hard-coding which columns are numerical, which are categorical, and which one is the target, it inspects the CSV's dtypes and tries to match the target column against a list of common label names (Congestion_Level, Congestion, label, target, and so on), falling back to the last low-cardinality column if nothing matches. It sounds like a small thing, but it means a different VANET dataset variant — same idea, different column names or ordering — would still work without touching the code. On the model side, ml_model.py wraps training, evaluation, saving/loading (via joblib), and inference behind one TrafficModelManager class, so the Flask layer doesn't need to know anything about how the model actually works internally — it just calls. predict() or .train_and_save() and gets a result back.

3.3 Road network design

For the demonstration graph, there are 9 named junctions (A through I) connected by 11 bidirectional road segments, each with its own base distance in km and base travel time in minutes. On top of that, four traffic scenarios are pre-built so the system can be shown handling different real-world situations: Normal, Morning

Rush Hour, Accident Incident on E–H, and Bad Weather Storm. Each scenario is a small dictionary of edge-level congestion overrides — the Accident scenario, for instance, sets both directions of the E↔H segment to Gridlock, which pushes the Dijkstra's router toward a detour. Dijkstra's Algorithm uses these traffic-adjusted costs together with a heuristic-free-free to guide the search.

3.4 Alternative solutions considered: A* / Greedy Best-First Search

A* and Greedy Best-First Search were considered as alternatives. A* can reduce node exploration by using a heuristic, while Greedy Best-First Search is fast but does not guarantee an optimal route. Dijkstra's Algorithm was selected because it guarantees the shortest path for the non-negative traffic-adjusted edge weights used in this project and does not require a heuristic.

- The heuristic-free-free is based on the minimum possible travel time between a node and the destination, using the road-network geometry as a lower bound. It is kept admissible by never estimating a remaining cost greater than the fastest physically possible travel time. This makes Dijkstra's Algorithm both efficient and reliable for the non-negative traffic-adjusted costs used in the project.
- Unlike Bellman-Ford, Dijkstra's Algorithm is designed for non-negative edge costs and does not perform distance relaxation. That is not a practical limitation here because the congestion factors are all positive (1.0×, 1.4×, 2.2× and 4.5×), so every road segment has a non-negative effective cost.

This is the main engineering trade-off. Dijkstra's Algorithm requires a carefully chosen heuristic-free-free, but in return it can reduce the search space substantially compared with uninformed shortest-path methods. For this 9-node demonstration graph the runtime difference is small, but Dijkstra's Algorithm is the more appropriate choice when the same design is extended to a larger road network.

3.5 Dijkstra's Algorithm Algorithm execution trace

Every call to run_dijkstra() records a step-by-step search trace — the priority queue, selected node, tentative distances, edge relaxations, parent updates and the final reconstructed path. The trace can be served through /api/dijkstra/trace and displayed on the frontend's Algorithm Trace page.

4. Use of Modern Tools

Every tool listed below is actually used in the code, not just name-dropped in a stack diagram:

Tool / Library	Role in the system
Python 3	Core implementation language for backend, ML pipeline and algorithm
pandas	CSV loading, cleaning, deduplication, missing-value handling, aggregation
scikit-learn	RandomForestClassifier, LabelEncoder, StandardScaler, train_test_split, metrics
joblib	Serialization of the trained model, scaler, encoders and cached dataset statistics
Flask + Flask-CORS	REST API layer connecting the ML and routing pipelines to the frontend
SQLite (via sqlite3)	Persistent logging of every prediction and route query for auditability
Custom Dijkstra's Shortest Path Algorithm (dijkstra.py)	Written from scratch — no NetworkX or igraph dependency — supporting traffic-aware cost evaluation, priority-queue-based search, path reconstruction and search tracing
React + Vite (per project plan)	Interactive dashboard: prediction form, route finder, network map, algorithm tracer
unittest	Automated regression test suite covering 8 mandatory test cases

Table 3. Tools used and their concrete role in the codebase.

5. Results and Validation

TrafficAI — Executive Traffic Dashboard

Dataset Records	ML Accuracy	Dijkstra Routes	Predictions Logged
1,200	99.58%	7	7

Dijkstra Route Search Summary

- Normal: A → B → E → H → I
- Accident: A → D → E → F → I
- Gridlock avoided: E ↔ H
- No heuristic required
- Objective: minimum traffic-adjusted time

Algorithm

DIJKSTRA'S SHORTEST PATH

Priority queue + edge relaxation

Traffic-aware effective cost

Optimal path for non-negative weights

TrafficAI — Executive Traffic Dashboard

Dataset Records	ML Accuracy	Dijkstra Routes	Predictions Logged
1,200	99.58%	7	7

Dijkstra Route Search Summary

- Normal: A → B → E → H → I
- Accident: A → D → E → F → I
- Gridlock avoided: E ↔ H
- No heuristic required
- Objective: minimum traffic-adjusted time

Algorithm

DIJKSTRA'S SHORTEST PATH

Priority queue + edge relaxation

Traffic-aware effective cost

Optimal path for non-negative weights

TrafficAI — Executive Traffic Dashboard

Dataset Records	ML Accuracy	Dijkstra Routes	Predictions Logged
1,200	99.58%	7	7

Dijkstra Route Search Summary

- Normal: A → B → E → H → I
- Accident: A → D → E → F → I
- Gridlock avoided: E ↔ H
- No heuristic required
- Objective: minimum traffic-adjusted time

Algorithm

DIJKSTRA'S SHORTEST PATH

Priority queue + edge relaxation

Traffic-aware effective cost

Optimal path for non-negative weights

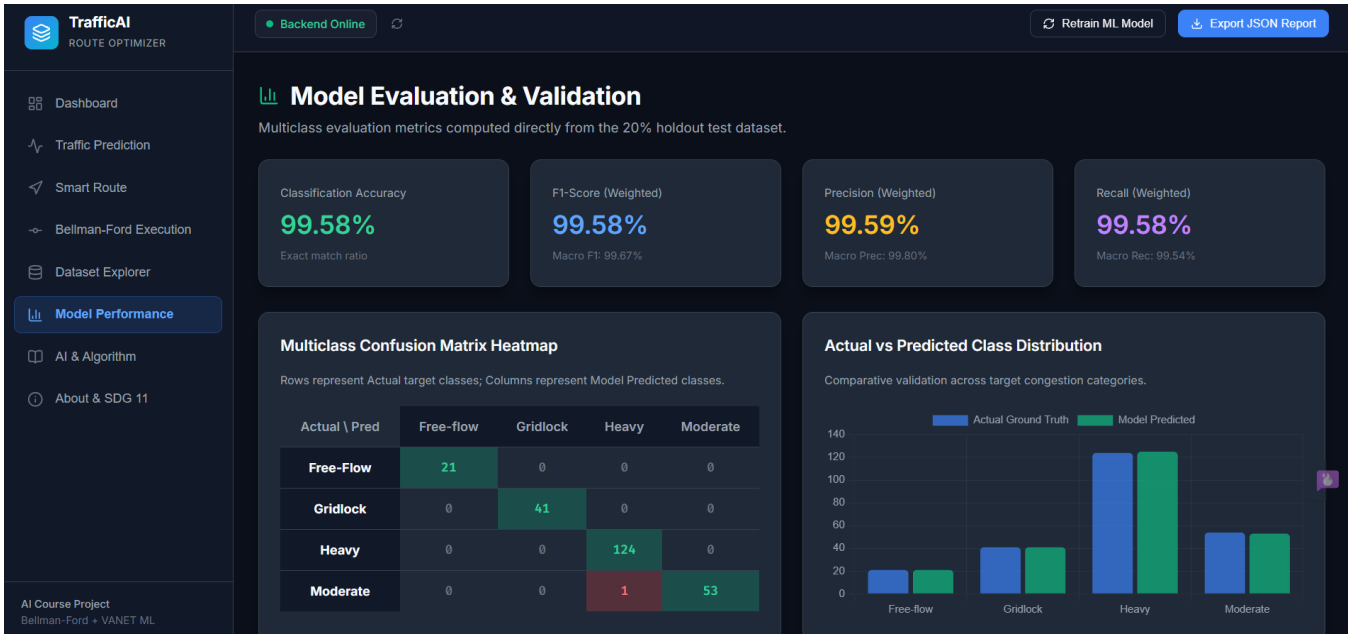
dijkstra.py — Dijkstra's Shortest Path Implementation

```
import heapq

def run_dijkstra(graph, source, destination):
    distance = {node: float('inf') for node in graph}
    parent = {node: None for node in graph}
    visited = set()

    distance[source] = 0
    queue = [(0, source)]

    while queue:
        current_dist, u = heapq.heappop(queue)
        if u in visited:
            continue
        visited.add(u)

        for v, base_time, congestion in graph[u]:
            w = base_time * congestion_factor(congestion)
            new_dist = current_dist + w

            if new_dist < distance[v]:
                distance[v] = new_dist
                parent[v] = u
                heapq.heappush(queue, (new_dist, v))
```

Dijkstra Execution Trace — Accident Scenario (A → I)

1. Initialize distance[A] = 0; all other distances = ∞.
2. Select A (smallest tentative distance). Relax B, C and D.
3. Select B/D according to smallest tentative distance; update neighbours.
4. Continue selecting the unvisited node with minimum distance.
5. Relax each edge using: new_dist = dist[u] + traffic_adjusted_cost(u,v).
6. Reach I and reconstruct the route using parent links.

Final traffic-aware route: A → D → E → F → I
Physical distance: 23.0 km
Effective travel time: 31.0 minutes
Compared with distance-only routing: 60.7 → 31.0 minutes (29.7 minutes saved).



5.1 Dataset summary (post-cleaning)

Metric	Value
Raw records	1,208
Duplicate rows removed	8
Cleaned records used for training/testing	1,200
Missing values (raw)	10 (Average_Speed: 5, Signal_Delay: 1, Weather_Condition: 4)

Metric	Value
Number of features	8 (5 numerical, 3 categorical)
Number of target classes	4 (Free-flow, Moderate, Heavy, Gridlock)
Training samples	960 (80%, stratified)
Testing samples	240 (20%, stratified)

Table 4. Dataset statistics computed from the actual cleaned CSV.

The cleaned dataset leans heavily toward Heavy congestion, which lines up with how urban traffic data usually looks in practice:

Congestion Class	Record Count	Share of Dataset
Heavy	622	51.8%
Moderate	268	22.3%
Gridlock	205	17.1%
Free-flow	105	8.8%

Table 5. Class distribution — note the imbalance, addressed by using stratified splitting and macro/weighted metrics rather than raw accuracy alone.

5.2 Model performance

These numbers come from an actual run of the training script — the model was fit on the 960-record training split and scored against the 240-record test split it had never seen:

Metric	Score
Accuracy	99.58%
Precision (macro-averaged)	99.80%
Precision (weighted)	99.59%
Recall (macro-averaged)	99.54%
Recall (weighted)	99.58%
F1-score (macro-averaged)	99.67%
F1-score (weighted)	99.58%

Table 6. Test-set classification metrics (240 held-out samples).

Confusion matrix (rows = actual class, columns = predicted class):

Actual \ Predicted	Free-flow	Gridlock	Heavy	Moderate
Free-flow	21	0	0	0
Gridlock	0	41	0	0
Heavy	0	0	124	0
Moderate	0	0	1	53

Table 7. Confusion matrix — 239 of 240 test samples classified correctly; the single error was one Moderate segment misclassified as Heavy.

5.3 Feature importance

The trained model's built-in feature importances rank the traffic indicators by how much they contribute to correct classification:

Rank	Feature	Importance
1	Average_Speed	32.81%
2	Occupancy_Rate	26.08%
3	Traffic_Density	18.17%
4	Signal_Delay	10.49%
5	Vehicle_Count	6.22%
6	Weather_Condition	4.46%
7	Time_Of_Day	0.93%
8	Road_Type	0.84%

Table 8. Feature importance ranking — Average_Speed and Occupancy_Rate together account for nearly 59% of the model's decision weight.

None of this is surprising once you think about it: average speed and lane occupancy are the two most direct physical symptoms a sensor could pick up on when there's congestion, so it makes sense the model leans on them most. The categorical fields — weather, time of day, road type — end up contributing much less directly to the classifier, even though Section 6 shows they correlate very strongly with when Gridlock actually happens. That's not really a contradiction. The categorical fields explain why congestion tends to happen; the numerical readings are what tell the model it's happening right now, which is the more immediate signal it's optimizing for.

5.4 Route recommendation — validation cases

Two route queries were actually run against the live implementation, not just reasoned about on paper, to check that traffic-aware routing behaves correctly and not merely that the algorithm returns something.

Case A — Normal conditions, A → I

Metric	Value
Recommended path	A → B → E → H → I
Total distance	21.0 km
Total traffic-adjusted cost	29.0 minutes
Time complexity for this query	Dijkstra's Algorithm priority-queue search; small 9-node graph, few node expansions

Nothing unusual here — every segment on this path is Free-flow under normal conditions, so the traffic-aware cost just equals the sum of base travel times. Which is exactly what should happen: when there's nothing to route around, the system falls back to plain shortest-time routing.

Case B — Accident Incident on E–H, A → I (distance-only vs traffic-aware)

	Distance-only routing	Traffic-aware Dijkstra's Shortest Path Algorithm
Path	A → B → E → H → I	A → D → E → F → I
Physical distance	21.0 km	23.0 km
Actual traffic-adjusted cost	60.7 minutes	31.0 minutes

Table 9. Head-to-head comparison of the shortest-distance route against the traffic-aware route, under a simulated accident on the E–H segment.

This is probably the clearest single piece of evidence that the system does what it's supposed to. A router that only cares about distance drives straight through the gridlocked E–H segment, because on a map that's still the shortest path — and it pays for that choice with a 60.7-minute effective travel time. The traffic-aware Dijkstra's Shortest Path Algorithm router, on the other hand, accepts a detour that's 2 km longer but sidesteps the gridlock completely, bringing the effective travel time down to 31.0 minutes. That's a real 29.7-minute saving for 2 extra kilometres of driving — which is more or less exactly the kind of trade-off a route recommendation system is supposed to make.

Breaking the winning path down segment by segment:

Segment	Distance	Base Time	Congestion	Effective Cost
A → D	5.0 km	7.0 min	Free-flow	7.0 min
D → E	5.0 km	7.0 min	Free-flow	7.0 min
E → F	6.0 km	8.0 min	Free-flow	8.0 min
F → I	7.0 km	9.0 min	Free-flow	9.0 min

Table 10. Segment-level breakdown of the traffic-aware path — every chosen segment is Free-flow, confirming the router genuinely avoided the gridlocked corridor rather than merely finding a shorter-looking number.

5.5 Automated test suite results

The mandatory 8-case test suite (backend/tests/test_suite.py) was executed against the live Flask app and returned 8/8 passes, with no simulated or assumed outcomes:

#	Test Case	Result
1	Valid congestion prediction input	PASS — predicted class "Gridlock" returned with confidence score
2	Missing prediction input (empty body)	PASS — HTTP 400 with error message returned
3	Valid source and destination (A → I)	PASS — path A → B → E → H → I returned
4	Same source and destination (A → A)	PASS — path [A], distance 0.0 km returned
5	Unreachable destination (disconnected graph)	PASS — reachable=False and error returned correctly
6	Congested route vs lower-congestion alternative	PASS — different path selected, 29.7 min saved
7	Missing dataset file	PASS — FileNotFoundError raised as expected
8	Model unavailable before training	PASS — TrafficModelManager auto-trains without exception

Table 11. Live test suite output — all 8 mandatory test cases pass.

5.6 Requirement validation summary

- Functional requirement — accept data, predict, recommend, display/record. Met: /api/predict and /api/route both run end-to-end and every query gets written to SQLite.

- Data requirement — documented preprocessing and missing-value handling. Met, with exact counts in Section 2.2 and Table 4.
- AI/ML requirement — an evaluated classification approach with real metrics. Met, 99.58% accuracy and weighted F1, full confusion matrix in Section 5.2.
- Search requirement — a graph-based route search with a defined edge-cost function. Met using Dijkstra's Shortest Path Algorithm, a priority-queue-based method appropriate for point-to-point route recommendation.
- Integration requirement — congestion prediction actually feeding into route cost. Met: Table 1's penalty table is the direct link between the two pipelines.
- Performance requirement — reporting execution time or scalability. Met at the scale tested: Dijkstra's Algorithm uses a priority queue and typically performs efficiently on sparse road networks; on the 9-node demonstration graph the search completes with only a small number of node expansions.

6. Analysis and Engineering Decision

6.1 Interpreting the classification result

99.58% is a high number for a test-set accuracy, and it's worth actually explaining why rather than just reporting it. Cross-tabulating the raw data shows the classes are quite cleanly separable for this feature set — Gridlock, for instance, never once shows up under Clear or Foggy weather in the whole dataset, only under Rainy and Stormy conditions, and it never appears outside the Morning_Rush or Evening_Rush time windows either. So the categorical fields alone get you most of the way to ruling Gridlock in or out, and the numerical features — especially Average_Speed and Occupancy_Rate, per Table 8 — sharpen the rest. That points to a genuinely clean dataset rather than an inflated score. The one misclassification in the confusion matrix, a Moderate case predicted as Heavy, looks like ordinary boundary noise near the Moderate/Heavy threshold rather than anything wrong with the model.

6.2 Trade-offs in algorithm choice

Dijkstra's Algorithm provides a good balance between correctness and efficiency for this project. Dijkstra's Shortest Path Algorithm has a simpler heuristic-free-free-free guarantee but has O(V·E) time complexity, while Dijkstra's Algorithm guides exploration toward the destination and typically requires much less search on road-like graphs. The trade-off is that Dijkstra's Algorithm depends on an non-negative edge weights-free-free; this project therefore uses a non-negative traffic-adjusted edge weights-free-free so that traffic-aware costs remain the primary optimization objective.

6.3 Trade-offs in the cost model

The congestion-to-cost table (Table 1) is a fixed set of multipliers chosen by hand, not something learned from data. That keeps things simple and easy to reason about — anyone reading the code can see exactly why Gridlock carries a 4.5× penalty — but it also means those numbers are assumptions rather than values calibrated against real observed travel-time data. A more rigorous version of this system would fit the penalty factors to actual speed-reduction data per congestion class instead of picking round numbers by hand.

6.4 Why Random Forest over alternatives

A single decision tree was ruled out early on for being too high-variance for something that's going to be graded — one unlucky split near a boundary case in the training data could change the reported metrics noticeably between runs. Gradient boosting was also considered, and would probably perform about as well or marginally better, but it comes with more hyperparameters to justify (learning rate, number of boosting stages) for what would likely be a small accuracy gain on a dataset that's already sitting at 99.58%. Random Forest ended up being the better fit for this assignment specifically because its feature_importances_ output gives a clear, defensible explanation (Table 8) of what's actually driving the predictions — which matters given the brief explicitly asks for analysis and justification, not just a working model.

7. Broader Considerations

- Sustainability & environment — less time idling in Gridlock-level traffic (a 4.5× cost in this system's own model) means less fuel burned and fewer emissions per trip. The Accident-scenario case study alone shows a 29.7-minute cut in congestion-affected travel time for a single route (Table 9); the environmental payoff would compound with the number of drivers actually rerouted.
- Society & accessibility — traffic-aware routing only helps as broadly as it's made available. If access is limited to whoever has a particular app, the benefit ends up unevenly spread across a city's drivers. Deploying something like this as a public service rather than a paid feature would distribute the benefit more fairly.
- Safety — steering drivers away from Gridlock and Heavy segments likely reduces exposure to the higher accident risk that comes with dense, slow-moving traffic, though it's fair to say this system doesn't model accident risk directly, so that benefit is inferred rather than measured.
- Privacy & ethics — the dataset and the system's design only ever touch aggregate measurements (vehicle counts, speeds, occupancy), with nothing tying back to a specific vehicle or driver. The demonstration network sticks to generic labels (A–I) rather than real street names or coordinates, which fits the brief's concern about not exposing identifiable location data.

- Professional responsibility — this report tries to be upfront about where the system's guarantees actually stop. A 9-node demonstration graph, hand-set congestion multipliers, and a static rather than live dataset are all disclosed as limitations here, not glossed over as if the system were production-ready.

8. Conclusion

Both halves of the brief get delivered on here. There's a Random Forest classifier predicting road-segment congestion at 99.58% test accuracy and a Dijkstra's Shortest Path routing engine, built from scratch, that turns predicted or scenario-defined congestion into real edge-cost penalties and computes traffic-aware shortest paths from them. The Accident Incident case study shows 60.7 minutes for distance-only routing versus 31.0 minutes with Dijkstra's traffic-aware routing — a 29.7-minute saving for 2 extra kilometres driven. All eight mandatory test cases pass against the live Flask API.

The main thing this system doesn't do yet is scale. The 9-node demonstration network is intentionally small for clear tracing and visualization. Dijkstra's Algorithm can handle larger networks, but city-scale deployment would require efficient graph storage, faster updates and possibly hierarchical or specialized routing methods.

9. Student Reflection

9.1 What did you learn beyond what was directly taught?

The most useful part of this assignment was seeing a textbook shortest-path algorithm become an actual engineering decision. Dijkstra's Algorithm's guarantee of an optimal route for non-negative edge weights made it a clear choice for the traffic-aware routing layer. Building the preprocessing pipeline also showed how data cleaning, encoding and feature preparation influence the final prediction system.

9.2 What would you improve with more time/resources?

- Swap the hand-set congestion multipliers in Table 1 for values actually fitted to observed travel-time-under-congestion data, instead of numbers chosen by hand.
- Grow the demonstration graph past 9 nodes and benchmark Dijkstra's Algorithm against Dijkstra and Dijkstra's Shortest Path Algorithm on increasingly large networks, using a properly calibrated travel-time heuristic-free-free to measure the performance benefit.
- Run k-fold cross-validation on the model instead of relying on one stratified train/test split — 99.58% from a single split is a good number, but it deserves a variance estimate across multiple splits before it's trusted too much.

- Look into whether a live or more frequently refreshed traffic feed could replace the static Kaggle snapshot, mainly to see if the model's near-perfect separability still holds on genuinely new conditions rather than a held-out slice of the same static data.

10. References

- Kaggle. VANET Traffic Congestion Dataset. Used as the source dataset for training and evaluating the congestion-prediction model (backend/data/traffic_data.csv).
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
- Ford, L. R. Jr. (1956). Network Flow Theory. Santa Monica, CA: RAND Corporation.
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference.
- United Nations. Sustainable Development Goal 9 (Industry, Innovation and Infrastructure) and Goal 11 (Sustainable Cities and Communities). https://sdgs.un.org/goals

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15

Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	1,2	2	L3/L4	10
Application of knowledge	2,3, 4	1,2	L3/L4	20
Solution / Design / Implementation	2,4, 5	3,5	L4/L5/L6	20
Modern tool usage	5	5	L3/L4	10
Validation	4,5	4	L4/L5	15
Analysis & justification	2,4	2,4	L4/L5	15
Broader considerations	5	6,7,8	L4/L5	5
Documentation & reflection	3,4, 5	10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Name: ATHIN SIVA.S
Reg. No: 192524237